

Table of Contents

1 Classe Request (Wrapper per Richieste HTTP).....	2
1.1 Panoramica.....	2
1.2 Architettura.....	2
1.3 Struttura della Classe.....	3
Proprieta'.....	3
Costruttore.....	3
Metodi Getter.....	3
1.4 Flusso della Richiesta HTTP.....	4
1.5 Esempi di Utilizzo.....	4
Esempio 1: Creazione dalla Richiesta Corrente.....	4
Esempio 2: Creazione Manuale per Testing.....	4
Esempio 3: Accesso ai Dati con Valori Predefiniti.....	5
1.6 Caratteristiche Principali.....	5
Type Safety.....	5
Ricerca Header NON Case-Sensitive.....	5
Valori Di Default.....	5
Immutabile.....	5
1.7 Diagramma del Flusso dei Dati.....	6
1.8 Strategia di Testing.....	6
1.9 Dettagli Implementazione Test.....	6
1.10 Gestione Errori e Caratteristiche di Sicurezza.....	7
Null Safety.....	7
Valori Predefiniti.....	7
Normalizzazione Tipi.....	7
1.11 Integrazione con il Sistema Router.....	7
1.12 Esempio Completo: Flusso Login.....	8
1.13 Considerazioni di Sicurezza.....	8
1.14 Considerazioni sulle Performance.....	9
1.15 Punti di Estensione.....	9
1.16 Esecuzione dei Test.....	9
1.17 Risultati Attesi dai Test.....	9

Documentazione del Sistema di Routing API - Componente Request

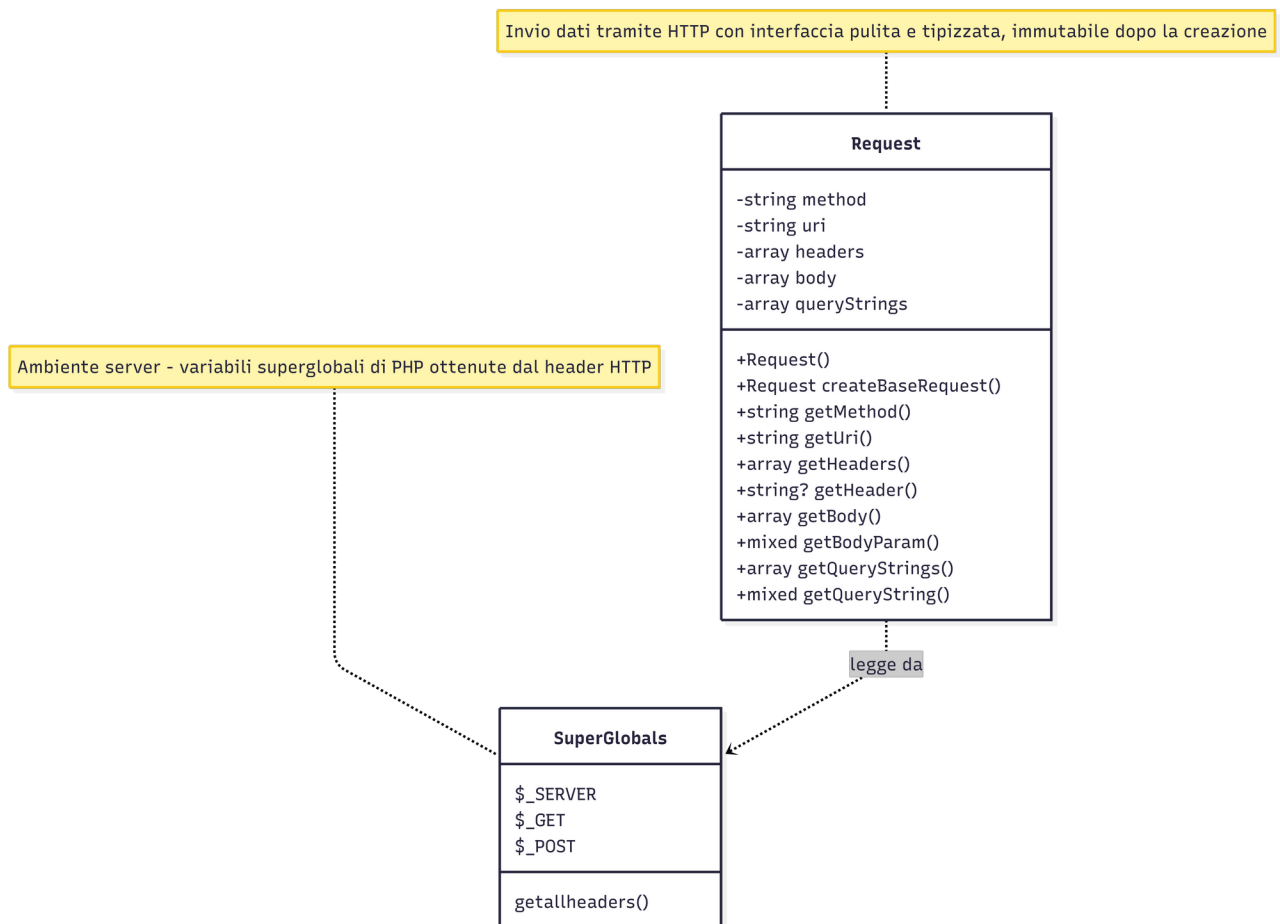
1 Classe Request (Wrapper per Richieste HTTP)

1.1 Panoramica

La classe **Request** incapsula i dati delle richieste HTTP in un semplice wrapper object-oriented.

Fornisce un'interfaccia pulita e tipizzata per accedere alle informazioni delle richieste HTTP senza interagire direttamente con le variabili globali di PHP (\$_SERVER[*]).

1.2 Architettura



1.3 Struttura della Classe

Proprieta'

Proprieta'	Tipo	Descrizione	Default
\$method	string	Metodo HTTP (GET, POST, PUT, ecc.)	'GET'
\$uri	string	URI della richiesta (percorso + query string)	'/'
\$headers	array	Header HTTP come coppie chiave-valore	[]
\$body	array	Parametri del corpo (POST e PUT)	[]
\$queryStrings	array	Parametri della query string (GET)	[]

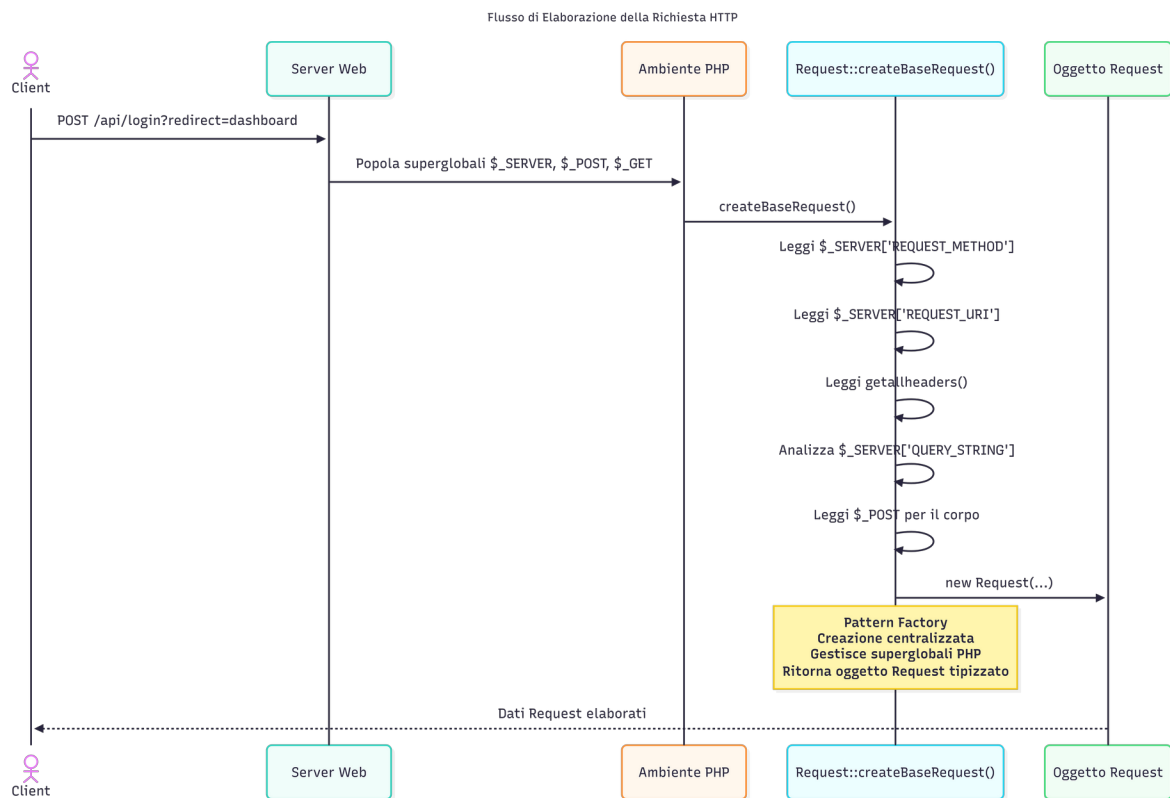
Costruttore

```
public function __construct(  
    string $method = 'GET',  
    string $uri = '/',  
    array $headers = [],  
    array $body = [],  
    array $queryStrings = []  
)
```

Metodi Getter

Metodo	Tipo ritornato	Descrizione
getMethod()	string	Metodo HTTP (gia' formattato in maiuscolo)
getUri()	string	URI della richiesta
getHeaders()	array	Tutti gli header
getHeader()	string null	Header specifico (case-insensitive)
getBody()	array	Tutti i parametri del corpo
getBodyParam()	mixed	Parametro specifico del corpo
getQueryStrings()	array	Tutti i parametri della query
getQueryString()	mixed	Parametro specifico della query

1.4 Flusso della Richiesta HTTP



1.5 Esempi di Utilizzo

Esempio 1: Creazione dalla Richiesta Corrente

```
// Crea dalla richiesta HTTP corrente
$request = Request::createBaseRequest();

// Accedi ai dati della richiesta
$method = $request->getMethod(); // 'POST', 'GET', ecc.
$uri = $request->getUri(); // '/api/login'
$contentType = $request->getHeader('Content-Type');
$username = $request->getBodyParam('username');
$page = $request->getQueryString('page', 1);
```

Esempio 2: Creazione Manuale per Testing

```
// Crea richiesta manualmente (utile per testing)
$request = new Request(
    'POST',
    '/api/utenti',
    ['Authorization' => 'Bearer token123'],
    ['nome' => 'Mario', 'email' => 'mario@esempio.com'],
    ['pagina' => '2', 'limite' => '20']
);
```

Esempio 3: Accesso ai Dati con Valori Predefiniti

```
// Accesso sicuro con valori predefiniti

// ritorna 1 se 'pagina' non presente
$pagina = $request->getQueryString('pagina', 1);

// Ritorna 'id' se 'ordina' non presente
$ordinamento = $request->getQueryString('ordina', 'id');

// Ritorna null se header non presente
$auth = $request->getHeader('Authorization');
```

1.6 Caratteristiche Principali

Type Safety

```
// Tutti i metodi hanno dichiarazioni di tipo
public function getMethod(): string
public function getHeader(string $name): ?string
public function getBodyParam(string $key, $default = null)
```

Ricerca Header NON Case-Sensitive

```
// Tutti questi ritornano lo stesso header
$header1 = $request->getHeader('content-type');
$header2 = $request->getHeader('Content-Type');
$header3 = $request->getHeader('CONTENT-TYPE');
```

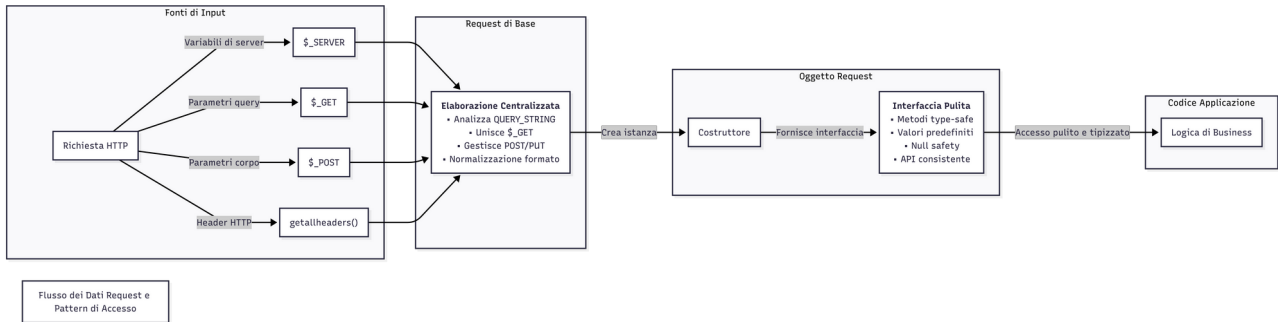
Valori Di Default

```
// Accesso sicuro con valori di fallback
$valore = $request->getBodyParam('opzionale', 'valore_predefinito');
$pagina = $request->getQueryString('pagina', 1);
```

Immutabile

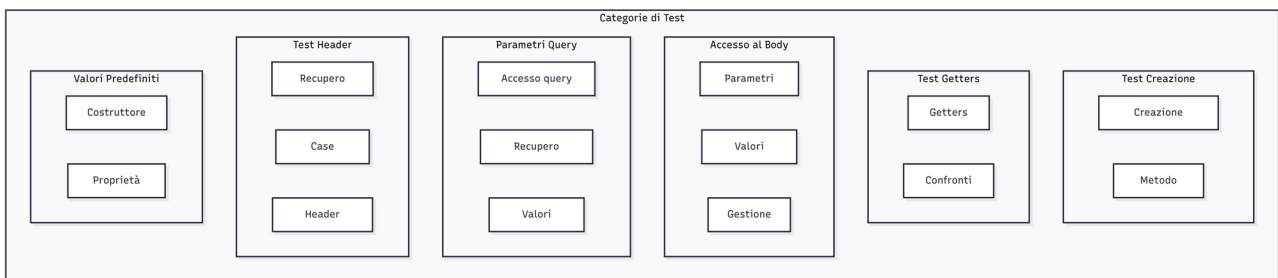
```
// Gli oggetti Request sono immutabili
$request = new Request('GET', '/api/dati');
// $request->setMethod('POST'); // NON POSSIBILE - non ci sono setter
```

1.7 Diagramma del Flusso dei Dati



1.8 Strategia di Testing

La classe RequestTest implementa testing completo:



1.9 Dettagli Implementazione Test

La classe RequestTest include 6 sezioni principali di test:

- 1 **Test della creazione di una Request di base** - Test del costruttore
- 2 **Test dei getters di una Request** - Test di tutti i metodi getter
- 3 **Test dell'accesso al corpo di una Request** - Test accesso parametri corpo
- 4 **Test dell'accesso dei parametri di una Request** - Test accesso parametri query
- 5 **Test dell'accesso degli headers di una Request** - Test recupero header
- 6 **Test dei valori di default di una Request** - Test valori predefiniti costruttore

Ogni test include:

- Descrizioni chiare dei
- Testing di asserzioni singole e multiple
- Reporting di successo/fallimento tramite stile CSS
- Gestione appropriata delle eccezioni

1.10 Gestione Errori e Caratteristiche di Sicurezza

Null Safety

```
// Ritorna null per header mancanti
$authHeader = $request->getHeader('Authorization'); // null se non presente

// Accesso sicuro parametri corpo
$opzionale = $request->getBodyParam('opzionale'); // null se non presente
```

Valori Predefiniti

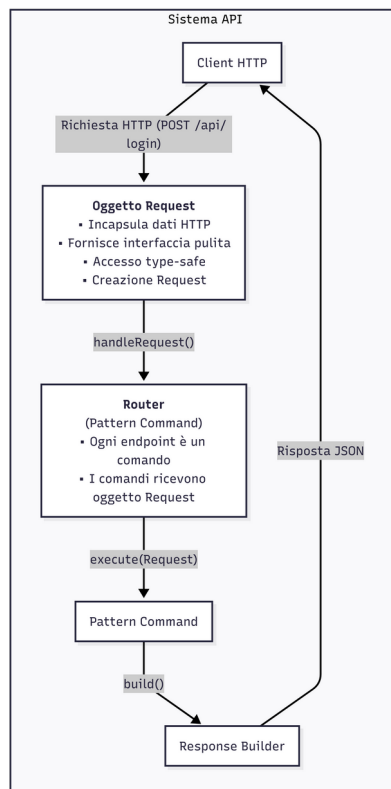
```
// Fornisce default sensati
$pagina = $request->getQueryString('pagina', 1); // Default: 1
$limite = $request->getQueryString('limite', 10); // Default: 10
$ordinamento = $request->getQueryString('ordina', 'id'); // Default: 'id'
```

Normalizzazione Tipi

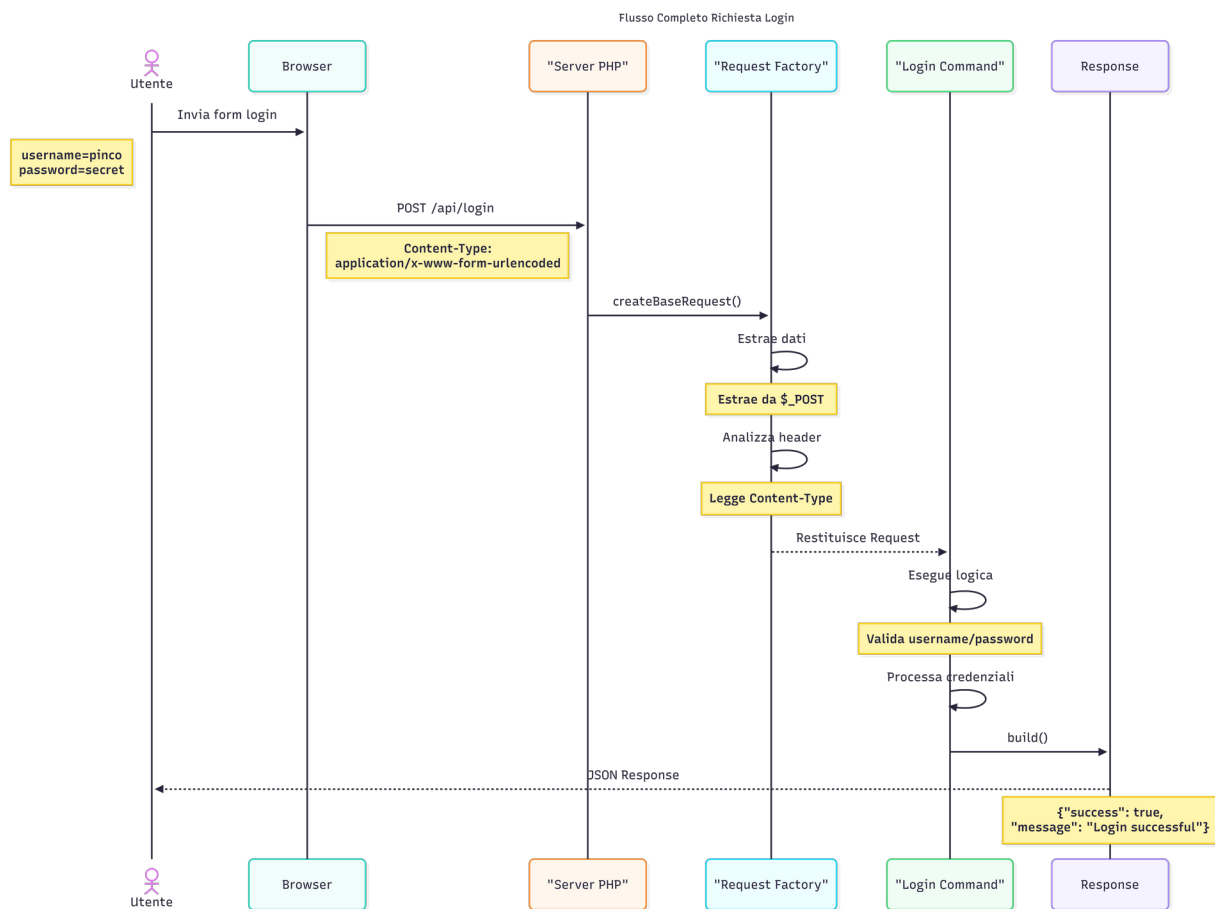
```
// Metodo HTTP sempre maiuscolo
$metodo = $request->getMethod(); // 'POST', 'GET', ecc.

// Ricerca header NON case-sensitive
$contentType = $request->getHeader('content-type');
```

1.11 Integrazione con il Sistema Router



1.12 Esempio Completo: Flusso Login



1.13 Considerazioni di Sicurezza

La classe Request include diverse decisioni di design consapevoli della sicurezza:

- 1 **Nessun Accesso Diretto alle Superglobali:** Il codice applicativo non tocca direttamente `$_POST`, `$_GET`
- 2 **Type Safety:** Previene vulnerabilità di confusione di tipo
- 3 **Header Case-Insensitive:** Previene injection di header tramite variazioni di case
- 4 **Design Immutabile:** Previene manipolazione della richiesta dopo la creazione
- 5 **Valori Predefiniti:** Gestione sicura dei parametri mancanti

1.14 Considerazioni sulle Performance

- 1 **Valutazione Lazy:** Gli header sono analizzati una volta nel metodo factory
- 2 **Copie Array:** I dati sono copiati dalle superglobali per l'immutabilit 
- 3 **Efficiente in Memoria:** Memorizza solo i dati necessari della richiesta
- 4 **Ricerca Veloce:** Accesso degli array per header e parametri

1.15 Punti di Estensione

La classe Request pu  essere estesa per funzionalit  aggiuntive ma non ne parleremo in questo documento poich  non richiesto.

1.16 Esecuzione dei Test

Per eseguire i test Response, una volta lanciata l'applicazione con Docker:

```
docker compose up --force-recreate --remove-orphans -d
```

Baster  navigare presso la pagina <http://localhost:8080/test/> e cliccare il link [Test Page for Response](#).

1.17 Risultati Attesi dai Test

Ogni test produrr  output simile a:

Test della classe Response con design pattern Builder

Test della creazione di una Response di base

- Creazione della Response tramite costruttore -

PASSED: `assertInstanceOf(Response)`

- Creazione della response tramite builder -

PASSED: `assertInstanceOf(Response)`

Con CSS styling che evidenzia:

- **Successo:** `Contorno verde`

- **Fallimento:** `Contorno rosso`